

Použití SQL v RPG

Vladimír Župka, 2025

Předkompilátor SQL pro jazyk RPG

Zdrojový typ **SQLRPGLE**

Kompilace **CRTSQLRPGI** – Create SQL ILE RPG Object

Volba **14** v PDM dosadí OBJTYPE (*PGM)

Volba **15** v PDM dosadí OBJTYPE (*MODULE)

OBJTYPE (*SRVPGM) nemá volbu v PDM

Source member

```
CRTSQLRPGI OBJ( STAOBR_SQL ) SRCFILE( QRPGLESRC ) SRCMBR( STAOBR_SQL )  
OBJTYPE( *PGM ) OUTPUT( *PRINT ) DBGVIEW( *SOURCE )
```

Stream file

```
CRTSQLRPGI OBJ( VZUPKA/STAOBR_SQL )  
SRCSTMF( ' /home/VZUPKA/STAOBR_SQL.SQLRPGLE' )  
OBJTYPE( *PGM ) OUTPUT( *PRINT ) DBGVIEW( *SOURCE )
```

Parametry překladu

COMMIT(<u>*CHG</u>)	*NONE *ALL ...
OBJTYPE(<u>*PGM</u>)	*MODULE *SRVPGM
OPTION(<u>*SYS</u> *SQL <u>*JOB</u> *SYSVAL *PERIOD *COMMA ...)	jmenná konvence
CLOSQLCSR(<u>*ENDACTGRP</u>)	*ENDMOD kdy se uzavře kurzor
DFTRDBCOL(<u>*NONE</u>)	předvolené SQL schema (default collection)
DYNDFTCOL(<u>*NO</u>)	*YES - DFTRDBCOL také pro dynamické příkazy
TOSRCFILE(<u>QTEMP/QSQLTEMP1</u>)	kam je uložen předkompilovaný zdrojový text
DBGVIEW(<u>*NONE</u>)	*SOURCE, *STMT, *LIST
...	

Zápis příkazů v RPG programu

Ve volném formátu

EXEC SQL *příkaz* ;

V pevném formátu

C/EXEC SQL příkaz

C/END-EXEC

Alternativní a doplňkové volby pro SQL příkazy. Některé jsou shodné s parametry předkompilátoru.

```
Exec SQL SET OPTION LANGID = CSY, SRTSEQ = *LANGIDSHR,  
          DATFMT = *ISO, COMMIT = *NONE ;
```

```

COMMIT = *CHG *NONE *ALL ...
NAMING = *SYS *SQL
DATFMT = *JOB *ISO ...
DATSEP = *JOB *PERIOD *COMMA *DASH *BLANK
DFTRDBCOL = *NONE
LANGID = *JOB *JOBRUN CSY ENU DEU ...
SRTSEQ = *JOB *HEX *JOBRUN *LANGIDUNQ *LANGIDSHR
TIMFMT = *HMS *ISO *EUR *USA *JIS
TIMSEP = *JOB *COLON *PERIOD *COMMA *BLANK
...

```

Soubory – tabulky

Soubor CENIKP – Ceník materiálu

A				UNIQUE
A	R	CENIKPF0		
A		<u>MATER</u>	5	COLHDG('Číslo' 'mater.')
A		CENA	10P 2	COLHDG('Cena/j.')
A		NAZEV	30	COLHDG('Název zboží')
A	K	MATER		

Soubor STAVYP – Stavý materiálových zásob

A				UNIQUE
A	R	STAVYPF0		
A		ZAVOD	2	COLHDG('Záv')
A		SKLAD	2	COLHDG('Skl')
A		<u>MATER</u>	5	COLHDG('Číslo' 'mater.')
A		MNOZ	10P 2	COLHDG('Množství')
A	K	ZAVOD		
A	K	SKLAD		
A	K	MATER		

Soubor OBRATP – Obraty materiálu

A	R	OBRATPF0		
A		ZAVOD	2	COLHDG('Záv')
A		SKLAD	2	COLHDG('Skl')
A		<u>MATER</u>	5	COLHDG('Číslo' 'mater.')
A		MNOBR	10P 2	COLHDG('Množství obratu')
A	K	ZAVOD		
A	K	SKLAD		
A	K	MATER		

Stavy a obraty – statické SQL příkazy SELECT, UPDATE

Program STAOB3_SQL

```
**free
```

```
Exec SQL set option COMMIT = *NONE;
```

```
Dcl-DS stavy ExtName('*LIBL/STAVYP') End-DS; // host variables
```

```
Dcl-DS obraty ExtName('*LIBL/OBRATP') qualified End-DS; // odstraní duplicitu
```

```
Exec SQL declare CS cursor for
```

```
    select ZAVOD, SKLAD, MATER, MNOZ from STAVYP;
```

```
Exec SQL open CS;
```

```
Exec SQL fetch from CS into :stavy;
```

```
dow sqlstate < '02000';
```

```
    Exec SQL update STAVYP set MNOZ = MNOZ +  
        ( select sum(MNOBR) from OBRATP  
          where ZAVOD = :ZAVOD  
            and SKLAD = :SKLAD  
            and MATER = :MATER  
          group by ZAVOD, SKLAD, MATER  
        )
```

```
    where ZAVOD = :ZAVOD and SKLAD = :SKLAD and MATER = :MATER;
```

```
    Exec SQL fetch from CS into :stavy;
```

```
enddo;
```

```
Exec SQL close CS;
```

```
return;
```

Chybové stavy

SQLSTATE obecnější		SQLCODE IBM
00000	Operace byla úspěšná , bez varování nebo výjimky.	0
01503	Počet výsledných sloupců je větší než poskytnutý počet proměnných.	+000, +030
02000	Nastala jedna z výjimek: - Výsledek příkazu SELECT INTO nebo subselektu v příkazu INSERT je prázdná tabulka. - Počet řádků určených v hledacím příkazu UPDATE nebo DELETE je nula. - Pozice kurzoru v příkazu FETCH je za posledním řádkem výsledné tabulky (konec dat jako EOF). - Orientace příkazu FETCH je nesprávná.	100
07001	Počet proměnných neodpovídá počtu parametrů (markerů)	-313
09000	Trigger v SQL příkazu selhal.	-723
42703	Zjištěn nedefinovaný sloupec nebo jméno parametru.	-205, -206, -213, -5001

Aktualizace příkazem MERGE

Program STAOBM_SQL

****free**

Exec SQL set option COMMIT = *NONE;

Exec SQL **MERGE INTO STAVYP S** // statický příkaz
 USING (select O.ZAVOD, O.SKLAB, O.MATER, sum(O.**MNOBR**) *SUMA_OBRATU*
 from **OBRATP** O
 group by O.ZAVOD, O.SKLAB, O.MATER
) as OBR
 ON (S.ZAVOD = OBR.ZAVOD
 and S.SKLAB = OBR.SKLAB
 and S.MATER = OBR.MATER)
 when MATCHED then
 update set MNOZ = MNOZ + OBR.*SUMA_OBRATU*
-- when NOT MATCHED then
-- insert (ZAVOD, SKLAB, MATER, MNOZ)
-- values(OBR.ZAVOD, OBR.SKLAB, OBR.MATER, OBR.*SUMA_OBRATU*)
;

return;

Stavy a obraty - RPG

Program STAOBR

```
**free
```

```
dcl-f STAVYP keyed usage(*update);  
dcl-f OBRATP keyed;
```

```
read OBRATP;  
dow not %eof(OBRATP);  
  chain (ZAVOD: SKLAD: MATER) STAVYP;  
  if %found;  
    MNOZ += MNOBR;  
    update STAVYPFO %fields(MNOZ);  
  endif;  
  read OBRATP;  
enddo;  
  
*inlr = *on;
```

Dynamický SELECT bez parametrů

Program DYNSEL

```
**free
```

```
Dcl-DS *N  ExtName('*LIBL/CENIKP')  End-DS;
```

```
Dcl-S spodni      packed(5: 2) inz( 2.5 );
```

```
Dcl-S horni       packed(5: 2) inz( 300 );
```

```
Dcl-S sel_stmt    char(500) inz;
```

```
sel_stmt = 'select MATER, CENAJ, NAZEV from CENIKP +  
           where CENAJ between ' + %char(spodni) + ' and ' + %char(horni) +  
           ' order by CENAJ asc for read only';
```

```
Exec SQL PREPARE PREP from :sel_stmt ;
```

```
Exec SQL declare CUR cursor for PREP;
```

```
Exec SQL open CUR ;
```

```
Exec SQL fetch CUR into :MATER, :CENAJ, :NAZEV;
```

```
DoW sqlstate < '02000';
```

```
    snd-msg *info MATER + ' ' + %editc(CENAJ: 'K') + ' ' + NAZEV;
```

```
    Exec SQL fetch CUR into :MATER, :CENAJ, :NAZEV;
```

```
EndDo;
```

```
Exec SQL close CUR;
```

```
return;
```

```
3 > dspjoblog
3 > call dynsel
00003      5.65  Prádelní šňůra
00002      9.40  Zubní pasta Kalodont
00001     12.40  PIŠKOTY OPAVIA
00004     14.90  Ponožky pánské tmavé
00006     14.95  Ponožky pánské bílé, nové
00012     48.30  Taška sportovní
00018     59.30  Husí sádlo v konzervě
00017    133.30  Hrábě dřevěné, kolíky plastové
00005    135.30  Tričko bílé
00011    162.30  Taška sportovní
00009    251.10  Whisky Balantine
```

Dynamický SELECT s markery

Program DYNSEL_MK

```
**free
```

```
Dcl-S sel_stmt      char(500) inz;  
Dcl-S spodni       packed(5: 2) inz( 2.5 );  
Dcl-S horni        packed(5: 2) inz( 300 );  
Dcl-DS *N  ExtName('CENIKP')  End-DS;
```

```
sel_stmt = 'select MATER, CENA, NAZEV from CENIKP +  
           where CENA between ? and ? +  
           order by CENA asc +  
           for read only' ;
```

```
Exec SQL PREPARE PREP from :sel_stmt ;  
Exec SQL declare CUR cursor for PREP;  
Exec SQL open CUR using :spodni, :horni;
```

```
Exec SQL fetch CUR into :MATER, :CENA, :NAZEV;
```

```
DoW sqlstate < '02000';
```

```
    snd-msg *info MATER + ' ' + %editc(CENA: 'K') + ' ' + NAZEV;
```

```
    Exec SQL fetch CUR into :MATER, :CENA, :NAZEV;
```

```
EndDo;
```

```
Exec SQL close CUR;
```

```
return;
```

Dynamický příkaz – EXECUTE IMMEDIATE

Program DYNEX_IM

```
**free
```

```
Dcl-S DYNST          Char(500) Inz;  
Dcl-S Limit          Packed(10: 2) Inz(500.00);
```

```
Exec SQL set option COMMIT = *NONE;
```

```
DYNST = 'update CENIKP set CENA = CENA * 1.10 +  
        where CENA < '  
DYNST = %trim(DYNST) + ' ' + %char(Limit) ;
```

```
Exec SQL EXECUTE IMMEDIATE :DYNST;
```

```
return;
```

Dynamický příkaz – PREPARE, EXECUTE, marker

Program DYNEX_MK

```
**free
```

```
Dcl-S DYNST          Char(500) Inz;
```

```
Dcl-PI *n;
```

```
    Limit Packed(10: 2); // parametr z CMD příkazu
```

```
End-PI;
```

```
Exec SQL set option COMMIT = *NONE;
```

```
DYNST = 'update CENIKP set CENA = CENA * 1.10  +  
        where CENA < ? ';
```

```
Exec SQL PREPARE STMT from :DYNST;
```

```
Exec SQL EXECUTE STMT using :Limit;
```

```
return;
```

CL příkaz k vyvolání programu DYNEX_MK LIMIT(250):

```
CMD          PROMPT('Volání SQL programu DYNEX_MK')
```

```
PARM          KWD(LIMIT) TYPE(*DEC) LEN(10 2) +  
              DFT(250) PROMPT('Limit ceny:')
```

Statické a dynamické příkazy

Statické příkazy jsou zapsány přímo v příkazu EXEC SQL

Dynamické příkazy jsou zapsány v textové proměnné

Postup příkazů pro SELECT

```
DECLARE SCROLL kurzor CURSOR FOR SELECT ...  
OPEN kurzor  
FETCH FIRST FROM kurzor INTO :proměnná, ... (NEXT, LAST, PRIOR, ...)  
CLOSE kurzor
```

Postup příkazů pro dynamický SELECT bez parametrů (markerů)

```
text-příkazu = 'SELECT ... '  
PREPARE připravený-příkaz FROM :text-příkazu  
DECLARE SCROLL kurzor CURSOR FOR připravený-příkaz  
... jako výše
```

Postup příkazů pro dynamický SELECT s parametry (markery)

```
text-příkazu = 'SELECT ... WHERE xyz BETWEEN ? AND ? ... '  
PREPARE připravený-příkaz FROM :text-příkazu  
DECLARE SCROLL kurzor CURSOR FOR připravený-příkaz  
OPEN kurzor USING :proměnná1, :proměnná2  
... jako výše
```

Postup příkazů pro ostatní dynamické příkazy bez parametrů (markerů)

```
text-příkazu = 'UPDATE ... SET ... '
```

```
EXECUTE IMMEDIATE :text-příkazu
```

Postup příkazů pro ostatní dynamické příkazy s parametry (markery)

```
text-příkazu = 'UPDATE ... SET ... WHERE xyz BETWEEN ? AND ?'
```

```
PREPARE připravený-příkaz FROM :text-příkazu
```

```
EXECUTE připravený-příkaz USING :proměnná1, :proměnná2
```


Lokální SQL tabulka v podproceduře

Program LOC_SQL

```
.....
**free
  // hlavní procedura volá podproceduru
ctl-OPT dftactgrp(*no); // kvůli podproceduře v modulu

  // volání podprocedury
dsply %editc(VRATIT_CENU('00001'): 'P'); // volání podprocedury
return;

.....

  // podprocedura vrací cenu pro číslo materiálu
dcl-PROC VRATIT_CENU export;

  dcl-DS cenik_ds extname('CENIKP') qualified end-DS;
  dcl-PI *N packed(10: 2); // rozhraní podprocedury
    material like(cenik_ds.MATER) CONST; // nebo VALUE
  end-PI;

  Exec SQL select CENA into :cenik_ds.CENA from CENIKP
        where MATER = :material;
  if sqlstate >= '02000';
    cenik_ds.CENA = -1;
  endif;
  return cenik_ds.CENA;
end-PROC;
.....
```

Lokální soubor v podproceduře

Program LOCFILE

```
.....
**free
  // hlavní program volá podproceduru
ctl-opt dftactgrp(*no);

  // volání podprocedury
dsply %editc( vratit_cenu('00001'): 'P');
return;

.....

  // podprocedura vrací cenu pro číslo materiálu
dcl-PROC VRATIT_CENU export;
  dcl-F CENIKP keyed; // příp. STATIC - nechá soubor otevřený
  dcl-DS cenik_ds likerec(CENIKPF0); // je nutná datová struktura

  dcl-PI *N packed(10: 2); // rozhraní podprocedury
    material like(cenik_ds.MATER) CONST; // nebo VALUE
end-PI;

chain(e) material CENIKP cenik_ds; // čte do datové struktury
if not %found();
  cenik_ds.CENAJ = -1;
endif;
return cenik_ds.CENAJ;
end-PROC;
.....
```

Dlouhá jména

Vytvoření SQL tabulky a naplnění záznamy

Program DL_JM1

```
**free
```

```
Exec SQL set option COMMIT = *NONE ;
```

```
Exec SQL CREATE OR REPLACE TABLE CENY_ZBOZI  
      ( CISLO_ZBOZI      CHAR(5)          UNIQUE,  
        CENA_ZA_JEDNOTKU DEC(12, 2),  
        NAZEV_ZBOZI      CHAR(50) CCSID 870  
      ) ;
```

```
Exec SQL INSERT INTO CENY_ZBOZI values ('00003', 1.25, 'Prádelní šňůra');
```

```
Exec SQL INSERT INTO CENY_ZBOZI values ('00004', 10.50, 'Ponožky pánské tmavé');
```

```
Exec SQL INSERT INTO CENY_ZBOZI values ('00005', 120.00, 'Tričko bílé');
```

```
Exec SQL INSERT INTO CENY_ZBOZI values ('00006', 10.55, 'Ponožky pánské bílé');
```

```
...
```

```
return;
```

V protokolu o kompilaci najdeme odpovídající systémová jména

První 4 znaky zůstávají, přidá se pořadové číslo.

```
...  
  
CENA_ZA_JEDNOTKU      * * * *      COLUMN  
                        8  
CENA_ZA_JEDNOTKU      6          COLUMN FOR CENA_00001 IN CENY_ZBOZI  
  
CENA_00001            6          DECIMAL(12,2) COLUMN IN CENY_ZBOZI  
  
...
```

Alternativně si můžeme volit vlastní systémová jména

```
Exec SQL CREATE OR REPLACE TABLE CENY_ZBOZI FOR SYSTEM NAME kratší-jméno  
      ( CISLO_ZBOZI FOR COLUMN SYSTEM NAME kratší-jméno CHAR(5),  
        CENA_ZA_JEDNOTKU FOR COLUMN SYSTEM NAME kratší-jméno DEC(12, 2),  
        NAZEV_ZBOZI FOR COLUMN SYSTEM NAME kratší-jméno CHAR(50)  
      );
```

Výpis cen zboží

Program DL_JM2

```
**free
```

```
Dcl-DS *N ExtName('CENY_ZBOZI') End-DS; // proměnné – host variables
```

```
Exec SQL declare CS cursor for
      select CISLO_ZBOZI, CENA_ZA_JEDNOTKU, NAZEV_ZBOZI
      from   CENY_ZBOZI
      order by CISLO_ZBOZI ;
```

```
Exec SQL open CS;
```

```
Exec SQL fetch from CS into :CISLO00001, :CENA_00001, :NAZEV00001 ;
```

```
dow sqlstate < '02000';
      snd-msg CISLO00001 + ' ' + %char(CENA_00001) + NAZEV00001;
      Exec SQL fetch from CS into :CISLO00001, :CENA_00001, :NAZEV00001 ;
enddo;
```

```
Exec SQL close CS;
```

```
return;
```


Plnění podsouboru pomocí SQL

Program STAPSZOB2

```
**free
dcl-f STAVYW2 workstn(*ext) usage(*input: *output) sfile(data: idx); // obraz. soubor
dcl-ds STAVYP extname('STAVYP') end-ds; // host variables
dcl-s idx int(10);

// získám data se souboru STAVYP
Exec SQL declare CURSOR cursor for
    select ZAVOD, SKLAD, MATER, MNOZ from STAVYP
    order by SKLAD, MATER asc for read only;
Exec SQL open CURSOR;

// naplním podsoubor z výsledku dotazu pomocí host variables
Exec SQL fetch from CURSOR into :ZAVOD, :SKLAD, :MATER, :MNOZ ;
dow not %eof;
    idx += 1;
    write DATA; // zapíše nový záznam do podsouboru podle pořadového čísla
    Exec SQL fetch from CURSOR into :ZAVOD, :SKLAD, :MATER, :MNOZ ;
enddo;
Exec SQL close CURSOR;

*in51 = *on;
exfmt RIDICI;

if *in03;
    *inlr = *on; // uvolním obraz. soubor
    return;
endif;
```

Plnění podsouboru bez SQL – datové struktury

Program STAPSZOBDS

```
dcl-f STAVYP keyed; // disk(*ext) usage(*input)
dcl-f STAVYW2 workstn(*ext) usage(*input: *output) sfile(data: idx);

dcl-ds RID_DS likerec(RIDICI: *all);
dcl-ds DAT_DS likerec(DATA: *all);
dcl-s idx int(10);

read STAVYP read_ds; // přečte ze souboru do datové oblasti
dow not %eof;
  idx += 1;
  eval-corr DAT_DS = read_ds; // EVAL-CORR - přesun podle jmen podpolí
  write DATA DAT_DS; // zapíše do podsouboru z datové oblasti
  read STAVYP read_ds;
enddo;

RID_DS.in51 = *on; // indikátory jsou v datové oblasti
exfmt RIDICI RID_DS; // EXFMT s datovou strukturou

if RID_DS.in03;
  *inlr = *on; // uvolním obraz. soubor
  return;
endif;
```


Datové typy RPG a SQL

RPG	SQL
CHAR(n)	CHAR(n)
VARCHAR(n:2)	VARCHAR(n)
UCS2(n)	GRAPHIC(n)
VARUCS2(n:2)	VARGRAPHIC(n)
PACKED(n:m)	DECIMAL(n, m)
ZONED(n:m)	NUMERIC(n, m)
INT(5)	SMALLINT
INT(10)	INTEGER
INT(20)	BIGINT
BINDEC(1-4:0)	SMALLINT
BINDEC(5-9:0)	INTEGER
FLOAT(4)	FLOAT(24) REAL
FLOAT(8)	FLOAT(53) FLOAT
DATE(*DMY-)	DATE
TIME(*HMS.)	TIME
TIMESTAMP(n)	TIMESTAMP(n)

Neodpovídající typy

```
dcl-S bin_1      SQLTYPE(BINARY: 50);      // CHAR(50) CCSID(*HEX);  
dcl-S varbin_1  SQLTYPE(VARBINARY: 100);   // VARCHAR(100) CCSID(*HEX);
```

```
dcl-S FIELD_1   SQLTYPE (CLOB: 1000 );      // DCL-DS FIELD_1;  
                                              //      FIELD_1_LEN  UNS(10);  
                                              //      FIELD_1_DATA CHAR(1000);  
                                              // END-DS FIELD_1;
```

```
Exec SQL set :FIELD_1 = 'ABCD';             // příkaz SQL v RPG  
FIELD_1_DATA = 'ABCD';                      // příkaz v RPG
```

Trigger – zvýšení ceny nejvýše o 10% při aktualizaci záznamu

Trigger externí – RPG program

Program TG_CENA_U

****free**

```
Dcl-PI *n;      // vstupní parametry z databázového systému
  Buf          LikeDS(TrgBuffer);
  Len          Uns(10); // nepotřebuji
End-PI;
```

```
      // Trigger buffer (1. parametr) – obsahuje data nového záznamu
Dcl-DS TrgBuffer          Len(1000);
  ...
  OldRecOffset            Uns(10) pos(49);
  OldRecLen               Uns(10);
  NewRecOffset            Uns(10) pos(65);
  NewRecLen               Uns(10);
End-DS;
```

```
// Starý záznam (before update)
Dcl-DS OldRecord ExtName('CENIKP') Based(OldRecPtr) Qualified End-DS;
// Nový záznam (before insertion)
Dcl-DS NewRecord ExtName('CENIKP') Based(NewRecPtr) Qualified End-DS;
```

```
Dcl-S OldRecPtr          Pointer; // Prázdný ukazatel na Starý záznam
Dcl-S NewRecPtr          Pointer; // Prázdný ukazatel na Nový záznam
```

```
OldRecPtr = %Addr(Buf) + Buf.OldRecOffset; // Adresa Starého záznamu
NewRecPtr = %Addr(Buf) + Buf.NewRecOffset; // Adresa Nového záznamu

If (NewRecord.CENA > 100 and NewRecord.CENA > OldRecord.CENA * 1.10) ;
    NewRecord.CENA = OldRecord.CENA * 1.10 ;
EndIf;

Return;
```

Trigger externí – registrace

```
AddPfTrg FILE(CENIKP) TRGTIME(*BEFORE) TRGEVENT(*UPDATE) PGM(TG_CENA_U)
        RPLTRG(*YES) TRG(TG_CENA_U) TRGLIB(*FILE) ALWREPCHG(*YES)
```

Odstranění

```
RmvPfTrg FILE(CENIKP) TRGTIME(*BEFORE) TRGEVENT(*UPDATE)
        TRG(TG_CENA_U) TRGLIB(*FILE)
```

Smazání

```
DROP TRIGGER TG_CENA_U
```

Uložená procedura – hodnota všeho materiálu ve skladě

Procedura externí – definiční skript

Skript PR_CEN_E

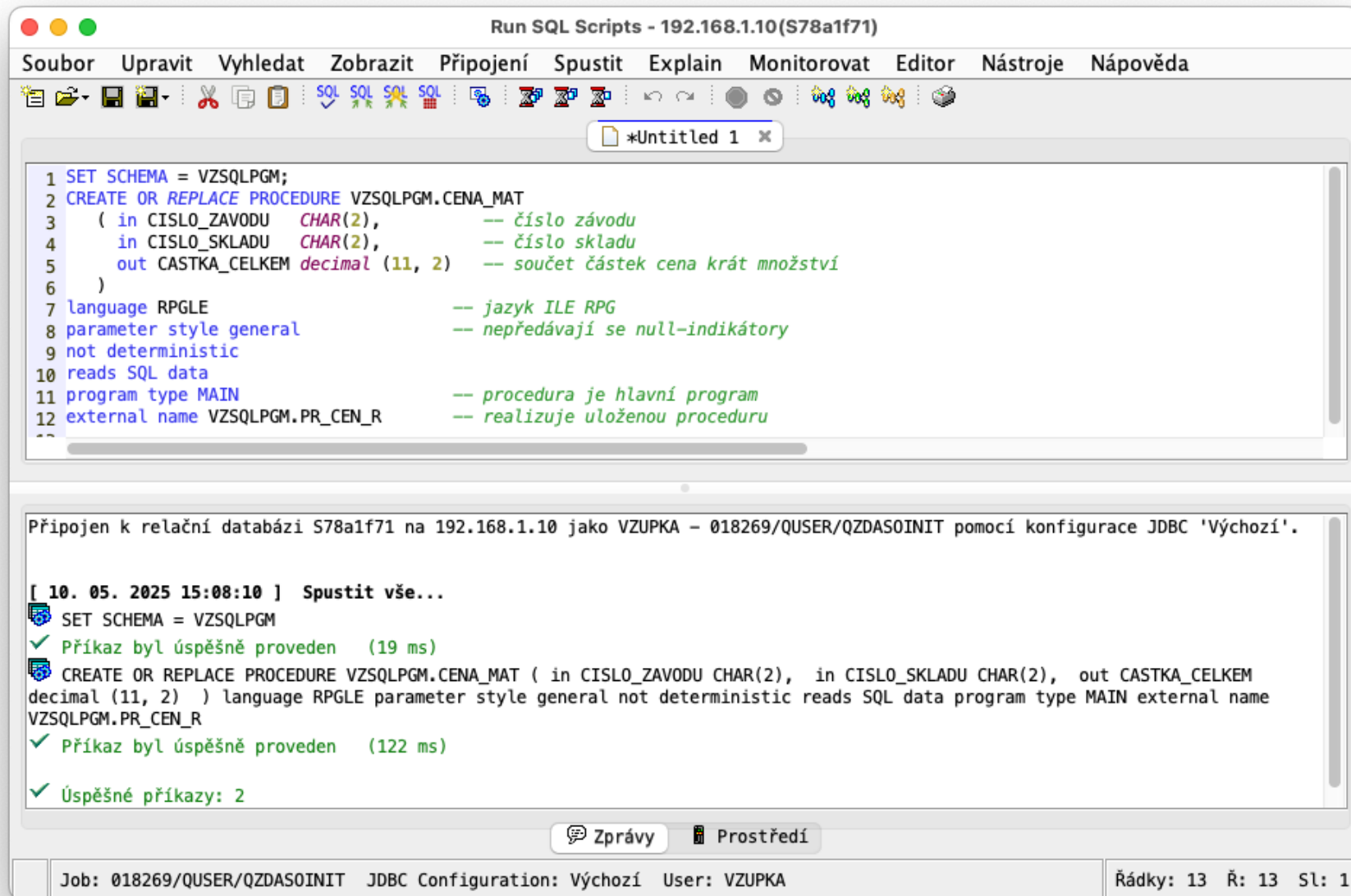
```
SET SCHEMA = VZSQLPGM;  
CREATE OR REPLACE PROCEDURE VZSQLPGM.CENA_MAT  
  ( in CISLO_ZAVODU   CHAR(2),      -- číslo závodu  
    in CISLO_SKLADU   CHAR(2),      -- číslo skladu  
    out CASTKA_CELKEM decimal (11, 2) -- součet částek cena krát množství  
  )  
language RPGLE                                -- jazyk ILE RPG  
parameter style general                       -- nepředávají se null-indikátory  
not deterministic  
reads SQL data  
program type MAIN                            -- procedura je hlavní program  
external name VZSQLPGM.PR_CEN_R              -- realizuje uloženou proceduru
```

Umístíme skript do zdrojového členu PR_CEN_E a spustíme CL příkazem

```
RUNSQLSTM SRCFILE(QSQLSRC) SRCMBR(PR_CEN_E) MARGINS(160)
```

nebo

V aplikaci **ACS** (IBM i Access Client Solutions) vložíme skript do okna **Run SQL Scripts** a spustíme.



Procedura externí – RPG program (s SQL)

Program VZSQLPGM/PR_CEN_R

```
**free
```

```
    // Parametry uložené procedury
```

```
Dcl-PI *n;
```

```
    Zavod                Char(2);      // in
```

```
    Sklad                Char(2);      // in
```

```
    Suma                Packed(11:2);  // out
```

```
End-PI;
```

```
Exec SQL select sum(CENA * MNOZ) into :Suma      -- out
        from STAVYP as S
        join CENIKP as C on C.MATER = S.MATER
        where ZAVOD = :Zavod and SKLAD = :Sklad ;
```

```
Return;
```

Procedura externí – RPG program (bez SQL)

Program VZSQLPGM/PR_CEN_R2

```
**free
```

```
      //   Parametry uložené procedury  
Dcl-PI *n;  
      Zav          Char(2);          // in  
      Skl          Char(2);          // in  
      Suma         Packed(11:2);     // out  
End-PI;
```

```
Dcl-F STAVYP keyed;  
Dcl-F CENIKP keyed;
```

```
Read STAVYP;  
Dow not %eof;  
    Chain (MATER) CENIKP;  
    If %found and ZAVOD = Zav and SKLAD = Skl;  
        Suma += MNOZ;    // out  
    Endif;  
    Read STAVYP;  
Enddo;  
*inlr = *on;
```


Volání procedury v programu

Program VZSQLPGM/PR_CEN_C

```
Dcl-F QPRINT    Printer(120);
```

```
Dcl-PI *n; // vstupní parametry uložené procedury
```

```
    Zavod                Char(2);
```

```
    Sklad                Char(2);
```

```
End-PI;
```

```
Dcl-S Suma                Packed(11:2); // výstupní parametr
```

```
Exec SQL  CALL  CENA_MAT ( :Zavod, :Sklad, :Suma );
```

```
Except DETAIL;    // tisk výsledků
```

```
*inlr = *on;      // uzavře soubor QPRINT
```

OQPRINT	E	DETAIL	1	
O				'Celková cena materiálu '
OQPRINT	E	DETAIL	1	
O				'Závod: '
O		Zavod	+1	
OQPRINT	E	DETAIL	1	
O				'Sklad: '
O		Sklad	+1	
OQPRINT	E	DETAIL	1	
O				'Celkem: '
O		Suma	P	+1

CL příkaz k vyvolání programu PR_CEN_C

```
CMD      PROMPT('Volání SQL programu PR_CEN_C')
PARM     KWD(ZAVOD) TYPE(*CHAR) LEN(2) +
          DFT('01') +
          PROMPT('Číslo závodu:')
PARM     KWD(SKLADE) TYPE(*CHAR) LEN(2) +
          DFT('01') +
          PROMPT('Číslo skladu:')
```

Výsledný tisk

```
Celková cena materiálu
Závod:  01
Sklad:  01
Celkem:      16966.62
```

User defined function (UDF) – vrací jednotkovou cenu

Funkce externí – definiční skript

Skript FU_CEN_E

```
SET SCHEMA = VZSQLPGM;  
CREATE OR REPLACE FUNCTION VZSQLPGM.VRAT_CENU_MATERIALU (MATERIAL CHAR(5))  
  RETURNS  DECIMAL (10, 2)  
  LANGUAGE  RPGLE  
  PARAMETER STYLE  GENERAL  
  NOT DETERMINISTIC  NO SQL  
  PROGRAM TYPE  SUB          -- funkci realizuje podprocedura  
  NOT FENCED          -- není vázána na stejnou úlohu  
  NO FINAL CALL        -- není první a poslední volání  
  EXTERNAL NAME VZSQLPGM.FU_CEN_R(FU_CEN_R) -- sevisní program a podprocedura
```

Umístíme skript do zdrojového členu FU_CEN_E a spustíme CL příkazem

```
RUNSQLSTM SRCFILE(QSQLSRC) SRCMBR(FU_CEN_E) MARGINS(160)
```

Funkce externí – RPG podprocedura

Program FU_CEN_R

```
**free
ctl-opt nomain;
dcl-PROC FU_CEN_R  export;
    dcl-F CENIKP keyed; // příp. STATIC – nechá soubor otevřený
    dcl-DS cenik_ds  likerec(CENIKPF0); // je nutná datová struktura

    dcl-PI *N  packed(10: 2); // rozhraní podprocedury
        material like(cenik_ds.MATER) CONST; // nebo VALUE
    end-PI;

    chain(e)  material  CENIKP  cenik_ds; // čte do datové struktury
    if not %found();
        cenik_ds.CENA = -1;
    endif;
    return cenik_ds.CENA;
end-PROC FU_CEN_R;
```

- Lokální soubor negeneruje popisy I a O s proměnnými.
- Pro datová pole je nutné použít **datovou strukturu** nebo funkci **%fields** u UPDATE.
- Soubor se otevře vždy při vstupu do podprocedury. Uzavírá se při výstupu z podprocedury a proměnné zanikají.
- Klíčové slovo STATIC u souboru nechá soubor otevřený a zachová jeho data pro příští volání.

Program s voláním funkce

Program FU_CEN_C

```
**free
```

```
Dcl-DS cenik_ds extname('CENIKP') qualified End-DS;
```

```
Dcl-S  cena_materialu Like(cenik_ds.CENAJ) Inz;
```

```
Dcl-PI *N;
```

```
    material Like(cenik_ds.MATER);  // vstupní parametr  
End-PI;
```

```
// volám SQL funkci
```

```
Exec SQL  set :cena_materialu = VRAT_CENU_MATERIALU (MATERIAL => :material );
```

```
Dsply cena_materialu;
```

```
Return;
```

Vytvořím **modul** pro servisní program

```
CRTTRPGMOD OBJ(FU_CEN_R) SRCFILE(QRPGLESRC)
```

Vytvořím **spojovací text** (binder source) v souboru QSRVSRC pro servisní program

```
STRPGMEXP SIGNATURE('VER1')
```

```
EXPORT SYMBOL(FU_CEN_R)
```

```
ENDPGMEXP
```

Vytvořím **servisní program**

```
CRTSRVPGM SRVPGM(FU_CEN_R) MODULE(*SRVPGM) SRCFILE(QSRVSRC) SRCMBR(*SRVPGM)
```

Vytvořím **modul** volajícího programu

```
CRTSQLRPGI OBJ(FU_CEN_C) SRCFILE(QRPGLESRC) SRCMBR(*OBJ) OBJTYPE(*MODULE)
```

Vytvořím **volající program** připojením servisního programu k modulu

```
CRTPGM PGM(FU_CEN_C) MODULE(*PGM) BNDSRVPGM((FU_CEN_R))
```

Spustím volající program testující UDTF funkci

```
CALL PGM(FU_CEN_C) PARM('00001')
```

User defined table function (UDTF) – vrací tabulku

UDTF externí – RPG podprocedura

Vytvoříme definiční **skript** pro externí uživatelskou funkci. Funkce OBJDAT_F přijímá dva parametry určující rozmezí datumů. Vybere objednávky z tabulky OBJHLA_T v daném rozmezí a **vytvoří tabulku** s čísly objednávek a s daty. Seznam bude uspořádán podle čísla objednávky vzestupně.

```
SET SCHEMA = VZSQL;  
CREATE OR REPLACE FUNCTION VZSQLPGM.OBJDAT_F (DATUM1 DATE, DATUM2 DATE)  
  RETURNS TABLE  
  ( COBJ CHAR(6) CCSID 870,  
    DTOBJ DATE )  
  LANGUAGE RPGLE  
  PARAMETER STYLE DB2SQL          -- umožňuje open, fetch, close  
  NOT DETERMINISTIC  
  READS SQL DATA  
  SCRATCHPAD                      -- průběžná paměť  
  PROGRAM TYPE SUB                -- funkce je ILE podprocedura  
  NOT FENCED                     -- není vázána na stejnou úlohu  
  NO FINAL CALL                  -- není první a poslední volání  
  CARDINALITY 1000               -- přibližné omezení počtu výsledných řádků  
  EXTERNAL NAME VZSQLPGM.OBJDAT_F(OBJDAT_F) -- serv. program a podprocedura
```

Skript umístíme do zdrojového členu, např. OBJDAT_FE a spustíme CL příkazem

```
RUNSQLSTM SRCFILE(QSQLSRC) SRCMBR(OBJDAT_FE) MARGINS(160)
```

UDTF externí – vytvoření funkce

Vytvořím zdrojový **modul** OBJDAT_F s procedurou (funkcí) OBJDAT_F_1. (Podprocedura nesmí mít stejné jméno jako servisní program.)

```
**free
Ctl-Opt nomain;
Dcl-Proc OBJDAT_F Export;
  Dcl-DS OBJHLA_T Ext Template ; // host variables z tabulky OBJHLA_T
End-DS;
// Datová struktura dat přetrvávajících mezi voláními funkce
Dcl-DS ScratchDS Template;
  ScrLen Int(10:0) Inz(%Size(ScratchDS));
  Cntr Packed(6:0) Inz(0);
End-DS;

// Procedure interface
Dcl-PI *N; // parametry
  Datum1 Date; // in
  Datum2 Date; // in
  COBJ_PAR Like(COBJ); // out
  DTOBJ_PAR Like(DTOBJ); // out
  Datum1_ind Int(5:0); // in
  Datum2_ind Int(5:0); // in
  COBJ_PAR_ind Int(5:0); // out
  DTOBJ_PAR_ind Int(5:0); // out
  SQLSTATE_PAR Char(5); // out
  Func_name VarChar(517); // in
  Spec_name VarChar(128); // in
  Message_text VarChar(1000); // out
  Scratchpad LikeDS(ScratchDS); // inout
  CallType Int(10:0); // in
End-PI;
```



```

If Calltype = -1;      // OPEN
  Exec SQL declare CUR cursor for
    select COBJ, DTOBJ
    from OBJHLA_T
    where DTOBJ between :Datum1 and :Datum2
    order by COBJ;
  Exec SQL open CUR;
ElseIf Calltype = 0;   // FETCH
  // Aby kurzor přetrval jednotlivá volání, je nutné při kompilaci
  // zadat parametr CLOSQLCSR(*ENDACTGRP)
  Exec SQL fetch next from CUR into
                                :COBJ_PAR :COBJ_PAR_ind,
                                :DTOBJ_PAR :DTOBJ_PAR_ind ;
ElseIf Calltype = 1;   // CLOSE
  Exec SQL close CUR;
EndIf;

End-Proc OBJDAT_F;

```

Poznámka 1: Čítač *Scratchpad.Cntr* není použit, mohl by sloužit např. k omezení nebo tisku počtu vrácených řádků.

Poznámka 2: Velmi důležitý je parametr CLOSQLCSR s hodnotou *ENDACTGRP zadaný při kompilaci modulu; zachovává otevřený kurzor mezi jednotlivými voláními procedury (Open, Fetch, Close). Kurzor se zavře až při ukončení aktivační skupiny. Předvolená hodnota *ENDMOD by způsobila, že kurzor by se zavřel po každém volání procedury (nejen při volání Close).

Poznámka 3: Jednotlivá volání jsou realizována jako vlákna (threads). V nich nejsou dostupné hodnoty proměnných pro výpis paměti (dump). Ladění je ale možné pomocí programu STRDBG.

Přijímá dva parametry a vrací tabulku obsahující čísla a data objednávek v rozmezí parametrů.

Vytvořím **spojovací text** (binder source) v souboru QSRVSRC **pro servisní program**

```
STRPGMEXP SIGNATURE( 'VER1' )  
EXPORT SYMBOL( OBJDAT_F )  
ENDPGMEXP
```

Vytvořím **servisní program** OBJDAT_F s procedurou (funkcí) OBJDAT_F_1

```
CRTSQLRPGI OBJ( VZSQLPGM/ OBJDAT_F ) SRCFILE( VZSQLPGM/QRPGLESRC ) SRCMBR( OBJDAT_F )  
OBJTYPE( *SRVPGM ) CLOSQLCSR( *ENDACTGRP )
```

Vytvořím **program** OBJDAT_P s připojeným servisním programem OBJDAT_F

```
CRTPGM PGM( VZSQLPGM/ OBJDAT_P ) MODULE( *PGM ) BNDSRVPGM( ( VZSQLPGM/ OBJDAT_F ) )
```

Program s voláním funkce

Program OBJDAT_P

****free**

Dcl-PI *n; // vstupní parametry

Dcl-S Datum1 Date;

Dcl-S Datum2 Date;

End-PI;

Dcl-S COBJ Char(6); // host variables pro sloupce tabulky

Dcl-S DTOBJ Date;

Exec SQL declare CUR cursor for

select * from **TABLE (OBJDAT_F(DATUM1 => :Datum1 , DATUM2 => :Datum2));**

Exec SQL open CUR ;

Exec SQL fetch CUR into :COBJ, :DTOBJ ;

DoW sqlstate = '00000';

SND-MSG COBJ + ' ' + %char(DTOBJ); // výpis do joblogu

Exec SQL fetch CUR into :COBJ, :DTOBJ ;

EndDo;

Exec SQL close CUR;

Return;

Interní SQL rutiny

Programovací jazyk SQL je popsán v dokumentaci

<https://www.ibm.com/docs/en/i/7.5.0?topic=reference-sql-procedural-language-sql-pl>,

<https://www.ibm.com/docs/en/i/7.5.0?topic=pl-sql-procedure-statement>.

Trigger interní – výkonný skript

Skript TG_CENA_U

```
-- Trigger BEFORE UPDATE pro tabulku CENIKP
CREATE OR REPLACE TRIGGER TG_CENA_U BEFORE UPDATE ON CENIKP
  REFERENCING OLD ROW AS old_row
                NEW ROW AS new_row
  FOR EACH ROW                                -- spustí se u každého řádku
  MODE DB2ROW                                  -- předepsáno pro BEFORE
  WHEN (new_row.CENAJ > 100 and
        new_row.CENAJ > old_row.CENAJ * 1.10 )
  BEGIN
    SET new_row.CENAJ = old_row.CENAJ * 1.10;
    SIGNAL SQLSTATE VALUE '01H01' SET MESSAGE_TEXT =
      'Navýšení množství přesahuje 10 %. Ponechá se navýšení 10 %';
  END ;
```

Umístíme skript do zdrojového členu TG_CENA_U a spustíme CL příkazem

```
RUNSQLSTM SRCFILE(QSQLSRC) SRCMBR(TG_CENA_U) MARGINS(160)
```

Smazání

```
DROP TRIGGER TG_CENA_U
```

Procedura interní – výkonný skript

Skript PR_CEN (v jazyku PL/SQL)

```
set schema = VZRPG_FREE;  
create or replace procedure VZSQLPGM.CENA_MAT  
  ( in CISLO_ZAVODU   CHAR(2),          -- číslo závodu  
    in CISLO_SKLADU   CHAR(2),          -- číslo skladu  
    out CASTKA_CELKEM decimal (11, 2)   -- Součet částek cena krát množství  
  )  
language SQL  
reads sql data  
begin  
  -- zjištění ceny materiálů v daném závodu a skladu  
  select sum(CENA * MNOZ) into CASTKA_CELKEM  
    from STAVYP as S  
   join CENIKP as C on C.MATER = S.MATER  
  where SKLAD = CISLO_SKLADU and ZAVOD = CISLO_ZAVODU ;  
end
```

Umístíme skript do zdrojového členu PR_CEN a spustíme CL příkazem

```
RUNSQLSTM SRCFILE(QSQLSRC) SRCMBR(PR_CEN) MARGINS(160)
```

nebo

v aplikaci **IBM i Access Client Solutions** vložíme skript do okna **Run SQL Scripts** a spustíme.

UDTF interní – výkonný skript

PL/SQL skript OBJDAT_F

Skript generuje uživatelskou funkci OBJDAT_F, která přijímá dva parametry a **vrátí tabulku** se dvěma sloupci, COBJ (číslo objednávky) a DTOBJ (datum objednávky). Objednávky vybere z tabulky OBJHLA_T v daném rozmezí.

```
SET SCHEMA = VZSQL;  
CREATE OR REPLACE FUNCTION VZSQLPGM.OBJDAT_F (DATUM1 DATE, DATUM2 DATE)  
RETURNS TABLE  
  ( COBJ CHAR(6) CCSID 870,  
    DTOBJ DATE )  
LANGUAGE SQL  
BEGIN  
  RETURN select COBJ, DTOBJ from OBJHLA_T  
         where DTOBJ between DATUM1 and DATUM2  
         order by COBJ;  
END
```

Skript umístíme do zdrojového členu, např. OBJDAT_F a spustíme CL příkazem

```
RUNSQLSTM SRCFILE(QSQLSRC) SRCMBR(OBJDAT_F) MARGINS(160)
```

Výsledkem skriptu je **servisní program OBJDA00001** v zadané knihovně. Ten je nutné spojit s modulem OBJDAT_P, aby vznikl **program OBJDAT_P**:

```
CRTPGM PGM(VZSQLPGM/OBJDAT_P) MODULE(*PGM) BNDSRVPGM((VZSQLPGM/OBJDA00001))
```

IBM i Services – příklady

SQL tabulky, pohledy, procedury nebo tabulkové funkce v knihovnách QSYS2 a SYSTOOLS nahrazují systémové funkce API. Snadno se používají v prostředí ACS. Lze je použít i v RPG.

Jejich přehled je v dokumentaci <https://www.ibm.com/docs/en/i/7.5.0?topic=optimization-i-services>.

Zjišťování údajů o objektech

Tabulková funkce **OBJECT_STATISTICS** funguje podobně jako CL příkazy DSPOBJD, RTVOBJD nebo API QUSROBJD. Parametry mají jména, která můžeme použít s oddělovačem =>. Nemusíme je používat vůbec. Popis funkce je [zde](#).

Volání tabulkové funkce ve skriptu

```
SELECT objname, objtype, objcreated, objsize
FROM TABLE ( QSYS2.OBJECT_STATISTICS
              (object_schema => 'VZUPKA',
               objtypelist => 'PGM, MODULE, SRVPGM',
               object_name => 'FU_CEN_*') )
```

Výsledek po spuštění skriptu v ACS:

OBJNAME	OBJTYPE	OBJCREATED	OBJSIZE
FU_CEN_C	*PGM	2025-03-29 11:30:31.000000	176128
FU_CEN_C	*MODULE	2025-03-29 11:21:46.000000	135168
FU_CEN_R	*MODULE	2025-03-29 11:30:12.000000	163840
FU_CEN_R	*SRVPGM	2025-03-29 11:30:25.000000	233472

Volání tabulkové funkce v RPG

Program OBJSTAT3

Výsledky tiskne a tiskový soubor převádí do IFS pomocí příkazu CPYSPLF.

```
dcl-f  QSYSPRT printer(120);
dcl-s  objname      varchar(10);
dcl-s  objtype      varchar(8) ;
dcl-s  objcreated   timestamp;
dcl-s  objsize      packed(15);
dcl-s  col_names    varchar(120);
dcl-s  columns      varchar(120);
dcl-s  cmdText      char(200);
dcl-s  cmdLen       packed(15: 5) inz(200);
dcl-pr cmdExc extpgm('QCMDEXC'); // prototyp volání programu QCMDEXC
      *n  char(200);
      *n  packed(15: 5);
end-pr cmdExc;

col_names = 'OBJNAME,OBJTYPE,OBJCREATED,OBJSIZE'; // jména sloupců
except header; // výstup jmen sloupců

Exec SQL declare CS cursor for
      SELECT objname, objtype, objcreated, objsize
      FROM TABLE ( QSYS2.OBJECT_STATISTICS
                    ('VZUPKA','PGM, MODULE, SRVPGM', 'FU_CEN_*) );
// čtení a tisk výsledků
Exec SQL open CS;
      Exec SQL fetch from CS
            into :objname, :objtype, :objcreated, :objsize ;
dow sqlstate < '02000';
      columns = '' + objname + ', ' + objtype + ', ' +
            %char(objcreated) + ', ' + %char(objsize); // hodnoty sloupců
      except detail; // výstup řádku s hodnotami sloupců
      Exec SQL fetch from CS
            into :objname, :objtype, :objcreated, :objsize ;
enddo;
```



```
Exec SQL close CS;
```

```
close QSYSPRT;
```

```
cmdText = 'CPYSPLF FILE(QSYSPRT) TOFILE(*TOSTMF) SPLNBR(*LAST) +  
          TOSTMF(''/home/vzupka/objstat3.txt'') STMFOPT(*REPLACE)';
```

```
cmdExc (cmdText: cmdLen);
```

```
return;
```

```
.....O*ilename++DF..N01N02N03Excnam++++B++A++Sb+Sa+.....Comments+++++
```

```
.....O*.....N01N02N03Field++++++YB.End++PConstant/editword/DTformat++Comments+++++
```

OQSYSPRT	E	header
O		col_names
OQSYSPRT	E	detail
O		columns

Tiskový výstup a obsah souboru /home/vzupka/objstat3.txt ve tvaru CSV:

```
OBJNAME,OBJTYPE,OBJCREATED,OBJSIZE
'FU_CEN_C','*PGM',2025-03-29-11.30.31.000000,176128
'FU_CEN_C','*MODULE',2025-03-29-11.21.46.000000,135168
'FU_CEN_R','*MODULE',2025-03-29-11.30.12.000000,163840
'FU_CEN_R','*SRVPGM',2025-03-29-11.30.25.000000,233472
```

Zobrazení v tabulkovém editoru Numbers

objstat3

OBJNAME	OBJTYPE	OBJCREATED	OBJSIZE
'FU_CEN_C'	'*PGM'	2025-03-29-11.30.31.000000	176128
'FU_CEN_C'	'*MODULE'	2025-03-29-11.21.46.000000	135168
'FU_CEN_R'	'*MODULE'	2025-03-29-11.30.12.000000	163840
'FU_CEN_R'	'*SRVPGM'	2025-03-29-11.30.25.000000	233472

Seznam schemat – funkce QSYS2.SYSSCHEMAS

Popis funkce je v publikaci <https://www.ibm.com/docs/en/i/7.5.0?topic=reference-sql> a konkrétně [zde](#).

```
-- List of schemas
SELECT cast (SCHEMA_NAME as varchar (10)), -- pomocí operátoru CAST
       varchar (SCHEMA_OWNER, 10) owner,   -- pomocí funkce VARCHAR
       cast (SCHEMA_CREATOR as varchar (10)),
       CREATION_TIMESTAMP as timestamp,
       SCHEMA_SIZE size,
       varchar (SCHEMA_TEXT, 20) schema_text,
       varchar (SYSTEM_SCHEMA_NAME, 10) sys_name,
       cast (IASP_NUMBER as smallint) as iasp
FROM QSYS2.SYSSCHEMAS
WHERE substring(SCHEMA_NAME, 1, 1) <> 'Q'
AND    substring(SCHEMA_NAME, 1, 3) <> 'SYS';
```

Skript v ACS

Run SQL Scripts - 192.168.1.10(S78a1f71)

SouborUpravitVyhledatZobrazitPřipojeníSpustitExplainMonitorovatEditorNástrojeNápověda

*Untitled 1 x

```
1  -- List of schemas
2  SELECT cast (SCHEMA_NAME as varchar (10)), -- pomocí operátoru CAST
3         varchar (SCHEMA_OWNER, 10) owner, -- pomocí funkce VARCHAR
4         cast (SCHEMA_CREATOR as varchar (10)),
5         CREATION_TIMESTAMP as timestamp,
6         SCHEMA_SIZE size,
7         varchar (SCHEMA_TEXT, 20) schema_text,
8         varchar (SYSTEM_SCHEMA_NAME, 10) sys_name,
9         cast (IASP_NUMBER as smallint) as iasp
10 FROM QSYS2.SYSSCHEMAS
11 WHERE substring(SCHEMA_NAME, 1, 1) <> 'Q'
12 AND  substring(SCHEMA_NAME, 1, 3) <> 'SYS';|
13
```

00001	OWNER	00003	TIMESTAMP	SIZE	SCHEMA_TEXT	SYS_NAME	IASP
#COBLIB	QSYS	QLPINSTALL	2024-04-24 22:22:37.000000	155 648 -		#COBLIB	0
#LIBRARY	QSYS	QLPINSTALL	2024-04-24 22:22:42.000000	114 688 -		#LIBRARY	0
#RPLIB	QSYS	QLPINSTALL	2024-04-24 22:22:37.000000	139 264 -		#RPLIB	0
IEDITOR	VZUPKA	VZUPKA	2025-09-04 12:31:55.000000	114 688	Code for i temporary	IEDITOR	0
KOLEKCE	VZUPKA	VZUPKA	2025-10-02 18:05:23.000000	163 840 -		KOLEKCE	0
MFA_EMKA	VZUPKA	VZUPKA	2024-12-30 14:33:32.000000	114 688 -		MFA_EMKA	0
MTCOBOL	QSECOFR	QSECOFR	2023-11-13 09:09:47.000000	147 456 -		MTCOBOL	0
MTRPG	QSECOFR	QSECOFR	2023-07-24 10:02:30.000000	131 072 -		MTRPG	0
MTRPGEX1	QSECOFR	QSECOFR	2024-04-25 09:50:27.000000	131 072	RPG Examples 1 (RPG	MTRPGEX1	0
MTRPGEX2	QSECOFR	QSECOFR	2024-05-03 11:25:12.000000	147 456	RPG Examples 2 (RPG	MTRPGEX2	0

Hotovo, počet načtených řádků: 42.05. 10. 2025 15:59:04

Zprávy ProstředíSELECT cast (SCHEMA_NAME as varchar (10)), varchar (SCHEMA_OWNER, 10) owner, cast (SCHEMA_CREATOR as...

Job: 036427/QUSER/QZDAS0INITJDBC Configuration: VýchozíUser: VZUPKARádky: 13Ř: 12Sl: 54

Funkce QSYS2.SYSSCHEMAS a zápis do IFS – procedura QSYS2.IFS_WRITE

Program SYSSCH_IFS

Dokumentace <https://www.ibm.com/docs/en/i/7.5.0?topic=optimization-i-services> speciálně [zde](#).

Sloupec `SCHEMA_TEXT` je ve výsledné tabulce definován jako `VARGRAPHIC(50) CCSID 1200`

Nullable, proto je uveden příkaz `ctl-opt CCSID(*GRAPH:*IGNORE);`

```
**free
```

```
ctl-opt CCSID(*GRAPH:*IGNORE);
```

```
dcl-s header varchar(200)
```

```
      inz('SCHEMA_NAME,SCHEMA_OWNER,SCHEMA_CREATOR,CREATION_DATE,+  
SIZE,SCHEMA_TEXT,SYS_NAME,IASP,');
```

```
dcl-s line   varchar(200);
```

```
dcl-ds *n;
```

```
      schema_name      varchar (10);
```

```
      owner            varchar (10);
```

```
      creator          varchar (10);
```

```
      timestamp        timestamp(1);
```

```
      size             packed (9: 0);
```

```
      label            varchar (30);
```

```
      schema_ind       int (5);
```

```
      sys_name         char (10);
```

```
      iasp             int (5);
```

```
end-ds;
```

```
dcl-s label2 varchar(30) inz(*all'-'); // nahrazuje hodnotu null
```

```
dcl-s iasp2 char(1);
```

Exec SQL

```
-- Create an empty IFS file
```

```
CALL QSYS2.IFS_WRITE(PATH_NAME => '/home/vzupka/schemas.csv',
```

```
      LINE => ' ',
```

```
      FILE_CCSID => 1208,
```

```
      OVERWRITE => 'REPLACE',
```

```
      END_OF_LINE => 'NONE');
```

Exec SQL

```
-- Add the header to the IFS file
CALL QSYS2.IFS_WRITE(PATH_NAME => '/home/vzupka/schemas.csv',
                    LINE => :header,
                    FILE_CCSID => 1208,
                    OVERWRITE => 'APPEND',
                    END_OF_LINE => 'CRLF');
```

Exec SQL declare CS cursor for

```
-- List of schemas
SELECT cast (SCHEMA_NAME as varchar (10)),
       varchar (SCHEMA_OWNER, 10) owner,
       cast (SCHEMA_CREATOR as varchar (10)),
       CREATION_TIMESTAMP as timestamp,
       SCHEMA_SIZE size,
       varchar (SCHEMA_TEXT, 20) schema_text,
       varchar (SYSTEM_SCHEMA_NAME, 10) sys_name,
       cast (IASP_NUMBER as smallint) as iasp
FROM QSYS2.SYSSCHEMAS
WHERE substring(SCHEMA_NAME, 1, 1) <> 'Q'
AND   substring(SCHEMA_NAME, 1, 3) <> 'SYS';
```

Exec SQL open CS;

DoW *on;

```
Exec SQL fetch from CS into :schema_name,
                          :owner,
                          :creator,
                          :timestamp,
                          :size,
                          :label :schema_ind,
                          :sys_name,
                          :iasp;

if sqlstate = '02000';
    leave;
endif;
if schema_ind = -1; // je hodnota sloupce null?
    label = label2;
endif;
```

```
iasp2 = %char(iasp);  // 1 character only
```

```
// construct text line
```

```
line = %trim(schema_name) + ',' +  
      %trim(owner) + ',' +  
      %trim(creator) + ',' +  
      %char(%date(timestamp)) + ',' +  
      %trim(%editc(size: 'P')) + ',' +  
      %trim(label) + ',' +  
      %trim(sys_name) + ',' +  
      iasp2 + ',';
```

```
Exec SQL
```

```
-- Add the text line to the IFS file
```

```
CALL QSYS2.IFS_WRITE(PATH_NAME => '/home/vzupka/schemas.csv',  
                     LINE => :line,  
                     FILE_CCSID => 1208,  
                     OVERWRITE => 'APPEND',  
                     END_OF_LINE => 'CRLF');
```

```
enddo;
```

```
Exec SQL close CS;
```

```
return;
```

Obsah souboru /home/vzupka/schemas.csv

```
SCHEMA_NAME,SCHEMA_OWNER,SCHEMA_CREATOR,CREATION_DATE,SIZE,SCHEMA_TEXT,SYS_NAME,IASP,
#COBLIB,QSYS,QLPINSTALL,2024-04-24,155648,-----,#COBLIB,0,
#LIBRARY,QSYS,QLPINSTALL,2024-04-24,114688,-----,#LIBRARY,0,
#RPGLIB,QSYS,QLPINSTALL,2024-04-24,139264,-----,#RPGLIB,0,
CORPDATA,VZUPKA,VZUPKA,2014-02-21,147456, COLLECTION - created,CORPDATA,0,
ILEDITOR,VZUPKA,VZUPKA,2025-09-04,114688,Code for i temporary,ILEDITOR,0,
KOLEKCE,VZUPKA,VZUPKA,2025-10-02,196608,-----,KOLEKCE,0,
MFA_EMKA,VZUPKA,VZUPKA,2024-12-30,114688,-----,MFA_EMKA,0,
MTCLP,QSECOFR,QSECOFR,2025-11-23,131072,-----,MTCLP,0,
...
```

Zobrazení v tabulkovém editoru Numbers

SCHEMA_NAME	SCHEMA_OWNER	SCHEMA_CREATOR	CREATION_DATE	SIZE	SCHEMA_TEXT	SYS_NAME	IASP	
#COBLIB	QSYS	QLPINSTALL	2024-04-24	155648	-----	#COBLIB	0	
#LIBRARY	QSYS	QLPINSTALL	2024-04-24	114688	-----	#LIBRARY	0	
#RPGLIB	QSYS	QLPINSTALL	2024-04-24	139264	-----	#RPGLIB	0	
CORPDATA	VZUPKA	VZUPKA	2014-02-21	147456	COLLECTION - created	CORPDATA	0	
ILEDITOR	VZUPKA	VZUPKA	2025-09-04	114688	Code for i temporary	ILEDITOR	0	
KOLEKCE	VZUPKA	VZUPKA	2025-10-02	196608	-----	KOLEKCE	0	
MFA_EMKA	VZUPKA	VZUPKA	2024-12-30	114688	-----	MFA_EMKA	0	
MTCLP	QSECOFR	QSECOFR	2025-11-23	131072	-----	MTCLP	0	